

Architecture

RAG SaaS Platform

Overview

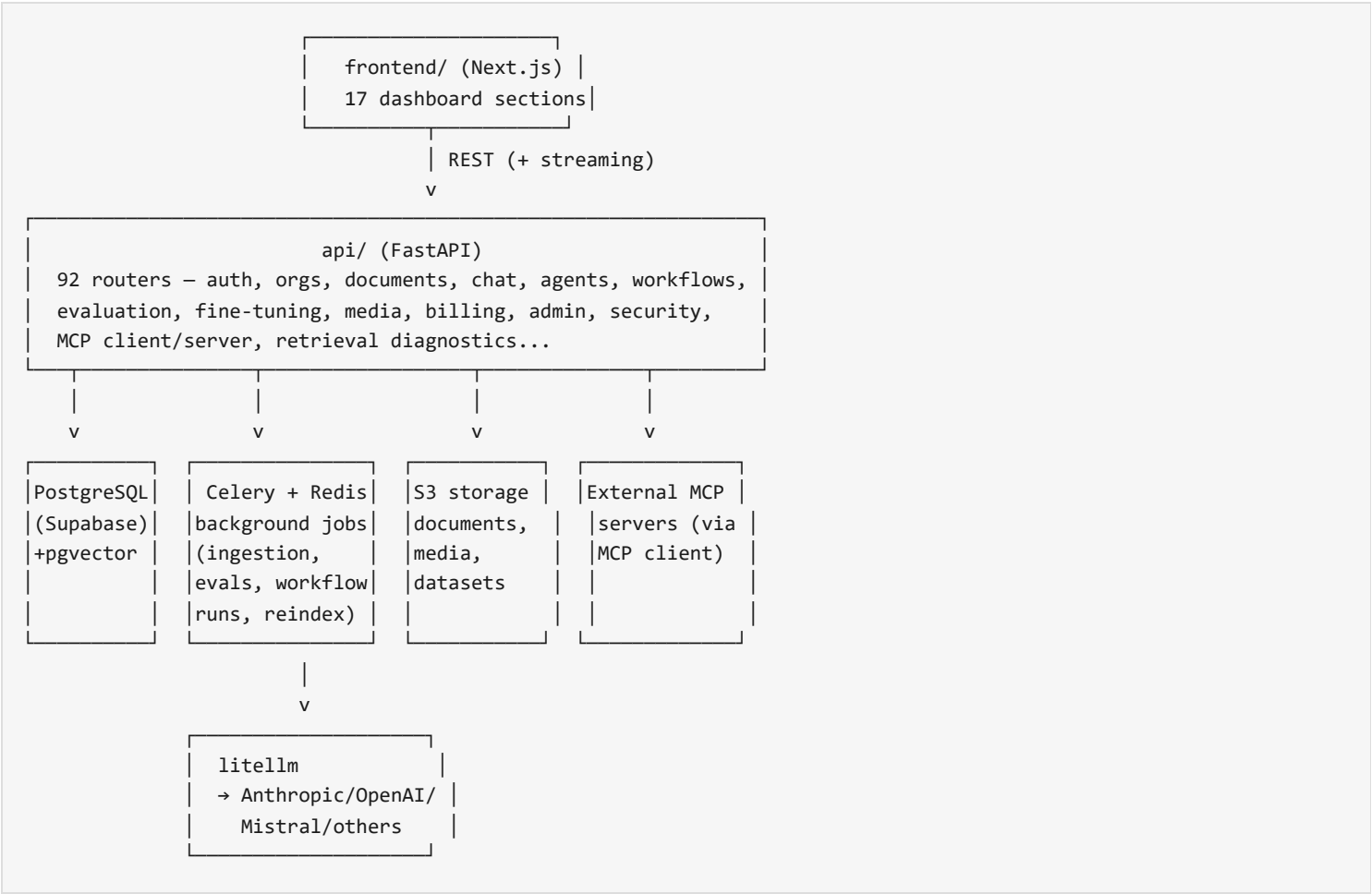
Architecture

System-level overview of the RAG SaaS Platform. For subsystem-level detail see [docs/advanced/](#) (RAG pipeline internals) and the per-feature docs linked throughout.

Subsystem deep-dives:

- [System overview](#) -- components, stack, flows
- [Data flow](#) -- RAG, agents, workflows end-to-end
- [Security model](#) -- multi-tenancy, RLS, permissions
- [Legacy status](#) -- src/ vs api/

High-level components



API layer (api/)

- **Framework:** FastAPI, async throughout.
- **ORM:** SQLAlchemy 2.0 async, migrations via Alembic

(api/alembic/versions/, head at migration 0108 as of Partie 24).

- **Auth:** session-based auth (`auth.py` , `password.py` , `two_factor.py` ,

webauthn.py, oauth.py, enterprise_sso.py), plus API keys for programmatic/agent access (agent_api_keys.py).

- **Multi-tenancy:** every tenant-scoped table carries an `org_id`, most

enforce row-level security (RLS) at the database level in addition to application-layer checks (`security/` modules per feature, e.g. `api/security/fine_tuning.py`, `api/security/autonomous_agents.py`).

- **Routers:** one module per feature area under `api/routers/`, each

paired with a `api/schemas/<feature>.py` (Pydantic request/response models), `api/security/<feature>.py` (access control), and usually `api/services/<feature>.py` (business logic) — see [docs/developer/API.md](#).

Background jobs (Celery + Redis)

Long-running or scheduled work never runs inline in a request: document ingestion, evaluation job execution, fine-tuning job polling, media processing (vision, YOLO, CLIP embedding), reindexing schedules, domain verification, backups. Task modules live under `api/tasks/`, one per feature area, mirroring the router/service split.

LLM access (litellm)

All LLM calls go through [litellm](#), which normalizes request/response shapes across Anthropic, OpenAI, Mistral, and other providers. This is what lets the platform support provider choice per organization, per agent, and per fine-tuning job without provider-specific call sites scattered through the codebase. Real, per-request cost tracking (`api/services/cost_tracking.py`) is built on top of litellm's reported token usage, and is reused directly by both autonomous-agent cost tracking (Partie 23) and evaluation-job cost reporting (Partie 7).

One deliberate exception: fine-tuning job submission talks to the OpenAI/Mistral fine-tuning REST APIs directly via `httpx` (`api/services/fine_tuning_providers.py`), because `litellm`'s own fine-tuning support doesn't cover Mistral. Anthropic has no public fine-tuning API at all, so fine-tuning jobs targeting Anthropic are rejected upfront with a clear `ProviderNotSupportedError` — a real, documented gap, not a silent failure. See [docs/fine-tuning/OVERVIEW.md](#).

RAG pipeline

The retrieval techniques were proven out in the original single-tenant demo (`src/`, see [src/README.md](#)) — hybrid BM25 + semantic search, Reciprocal Rank Fusion, cross-encoder reranking — and then rebuilt for multi-tenant scale in `api/services/`. See [docs/advanced/RAG_PIPELINE.md](#), [docs/advanced/CHUNKING.md](#),

[docs/advanced/EMBEDDINGS.md](#), [docs/advanced/RETRIEVAL.md](#), and [docs/advanced/RERANKING.md](#) for the current, multi-tenant implementation.

Workflow engine

`api/services/workflow_engine.py` is a real graph executor over a `Workflow`'s own `nodes / edges` (LLM call, RAG search, web search, HTTP call, condition, code, email, calendar, database, and a `human` approval block that pauses the run). Each real node execution is recorded in `WorkflowNodeExecution` (input/output/duration/status per step, mirroring `AgentTrace`'s own per-step tracing for agents) and streamed live over SSE (`GET /workflows/runs/{run_id}/stream`). See [docs/developer/API.md](#) and `docs/user/WORKFLOWS.md`.

Agent function calling and MCP

`AgentOrchestrator.run_agent` runs a real, provider-native function-calling loop (`tools=[...]` sent to the LLM, `tool_calls` parsed and executed, results fed back as `role:"tool"` messages, looped up to a bounded step cap). Tools come from three sources: the platform's own built-in registry (`api/services/tools.py`), an organization's custom webhook tools (`api/models/custom_tool.py`), and **MCP (Model Context Protocol)** — this platform is both an MCP *client* (agents call tools on an external MCP server an org registers, `api/services/mcp/client.py`, using Anthropic's own `mcp` SDK) and an MCP *server* (external MCP clients call this platform's own tool registry over `/mcp/v1/*`, reusing the same org-scoped API-key auth every other public endpoint uses). See [docs/mcp/README.md](#).

Cross-run agent memory (`AgentLongTermMemoryItem`, distinct from the existing per-session short-term memory) lets a tool persist a fact that outlives one conversation, org-wide or per-user.

Agents vs. autonomous agents

A real, deliberate architectural split, not two names for the same thing:

- **Agents** (`agents.py`, `custom_tools.py`) are configured chatbot

personas — a system prompt, a tool allowlist, model config — invoked per conversation turn, similar to how most RAG chat products expose "custom assistants."

- **Autonomous agents** (`autonomous_agents.py`,

`api/models/autonomous_agent.py`) run a real multi-step planning and execution loop toward a goal:

`AutonomousAgent` → `AgentPlan` → `AgentSteps`, with persistent `AgentMemory`, bounded `AgentCollaboration` between agents, guardrails, and cost limits. See [docs/autonomous/OVERVIEW.md](#).

Frontend (`frontend/`)

Next.js + React + TypeScript, tested with Vitest. One top-level dashboard section per major feature area under `frontend/app/dashboard/` (17 sections), each backed by its own `frontend/lib/services/.ts` *API client and frontend/Lib/hooks/.ts* data hooks, matching the API router it talks to. Design is intentionally plain: light theme only, no dark mode, an orange-and-white gradient, no decorative progress bars.

Evaluation (Eval Lab)

A real, mature evaluation system, not just the retrieval metrics it started from:

`EvaluationDataset` / `EvaluationQuestion` (with real ground-truth documents/answers), `EvaluationJob` (real, Celery-backed batch runs with cancellation), `EvaluationResult` (per-question Recall@K/MRR/NDCG/Precision/Hit-Rate, cost, latency), `ComparisonJob` (model/retriever/reranker/prompt comparison), `RegressionDetection` (automatic, per-organization regression alerts between two jobs), and `DeploymentEvaluation` (a real evaluation gate before promoting an agent version). See `docs/rag/` and `tests/test_evaluation_*.py`.

Developer platform (API keys, webhooks, SDKs)

External developers authenticate with an org-scoped API key (`OrganizationAPIKey`, hashed at rest, scoped, rate-limited, with rotation/expiration), get outbound **webhooks** for real platform events (`Webhook` / `WebhookDelivery`, HMAC-SHA256 signed, retried with backoff), and can use one of three real, tested SDKs shipped in this repo: [sdks/python/](#), [sdks/js/](#), and [sdks/react/](#) (a widget-embedding React component). See [docs/developer/API.md](#).

Observability

Beyond the OpenTelemetry/Loki/Grafana stack (`docker-compose.observability.yml`, off unless configured), the platform tracks retrieval quality and performance directly: `RetrievalDiagnostic` records the resolved strategy, final ranked chunks, and latency for every LIVE chat query (not just Eval Lab questions), `AgentTrace` / `WorkflowNodeExecution` give per-step execution history for agents and workflows, and `OrganizationUsage` tracks tokens/cost per organization. Sentry (`api/security/error_tracking.py`) captures backend + Celery errors when `SENTRY_DSN` is configured. See [docs/install/OBSERVABILITY_STACK.md](#).

Diagrams

Renderable Mermaid diagrams for this architecture, the database schema, the RAG pipeline, the auth flow, deployment topology, and multi-tenancy are in [docs/diagrams/](#).

Platform architecture

System Overview

Vue d'ensemble des composants, de la stack et des flux principaux. Pour le détail RAG, voir DATA_FLOW.md. Pour la sécurité, voir SECURITY.md. Pour le legacy src/, voir LEGACY.md.

Voir aussi le diagramme haut-niveau : ARCHITECTURE.md (racine).

Composants

Frontend (frontend/)

- Next.js (App Router), TypeScript, TailwindCSS.
- 17 sections dashboard : chat, documents, agents, workflows,

evaluation, fine-tuning, media, billing, admin, security, MCP...

- Branding dynamique : BrandingProvider + BrandingApplier.
- Streaming : SSE pour chat/agents.
- Widget embarquable : composant React autonome.

Backend (api/)

- FastAPI, async partout (SQLAlchemy 2.0 async).
- 92 routers.
- ORM : SQLAlchemy 2.0 async, migrations Alembic.
- Auth : sessions + API keys + SSO + 2FA + WebAuthn.
- LLM : abstraction via litellm, support Anthropic/OpenAI/Mistral + BYOK.

Background jobs (Celery + Redis)

- Worker Celery : ingestion, evals, workflow runs, reindex, notifications, backups.
- Transaction-mode pooler Supabase (port 6543).
- 28 fichiers de tâches migres vers api/tasks/_db.py.

Stockage

- PostgreSQL (Supabase) + pgvector.
- S3/R2 pour documents, media, datasets.
- Redis pour Celery broker + cache.

MCP (Model Context Protocol)

- Client : consomme des serveurs MCP externes.
- Serveur : expose les tools custom.

Stack technique

Couche	Technologie
Frontend	Next.js, TypeScript, TailwindCSS
Backend	FastAPI, Python 3.13, SQLAlchemy 2.0 async
DB	PostgreSQL (Supabase) + pgvector
Migrations	Alembic
Background	Celery + Redis
Storage	S3 / Cloudflare R2
LLM	litellm (Anthropic, OpenAI, Mistral, BYOK)
Auth	Sessions + API keys + SSO + 2FA + WebAuthn
Multi-tenant	RLS PostgreSQL + checks applicatifs

Flux principaux

Voir DATA_FLOW.md pour le detail :

- Ingestion de documents.
- Chat RAG.
- Agents.
- Workflows.

Securite

Voir SECURITY.md.

Deploiement

Voir docs/install/.

Data flow

Data Flow

Flux end-to-end des 4 pipelines principaux. Pour les composants, voir OVERVIEW.md. Pour le RAG detaillé, voir docs/advanced/RAG_PIPELINE.md.

1. Ingestion de documents

Upload (API) -> stockage S3/R2 + row Document (status=pending) -> tâche Celery dispatchée -> extraction (PDF/DOCX/HTML/... via parseurs dédiés) -> nettoyage (markdown, normalisation) -> chunking (sémantique, markdown, code, paragraphe...) -> embedding (provider configuré par org) -> insertion pgvector (chunks + métadonnées) -> Document.status = completed -> notification user (in-app + email)

Idempotence : reindex possible via Document.modification_check.

Points d'entrée : api/tasks/document_processing.py, api/services/document_extraction.py, api/services/chunking.py.

2. Chat RAG

User query (API ou SSE) -> auth + permission check -> résolution config (LLM provider, modèle, température) -> retrieval hybride (BM25 + vecteur pgvector) -> rerank (cross-encoder ou MMR) -> assemblage contexte + citations candidates -> LLM call (streaming ou non) -> post-traitement : citations persistées, confiance, groundedness -> stream tokens au client (SSE) -> débit crédits (tokens_input + tokens_output)

Points d'entrée : api/services/rag.py, api/services/generation.py, api/services/streaming.py, api/services/citations.py.

3. Agents

Agent run (API) -> création AgentRunRecord (status=running) -> chargement short-term memory (session) -> chargement long-term memory (cross-run, par agent+user) -> sélection outils (permission + pertinence) -> boucle LLM : LLM répond (texte ou tool_calls) -> si tool_calls : validation, exécution, feedback role:tool, reprise -> si réponse finale : break -> guardrails (validation finale) -> write-back long-term memory (best-effort, LLM extraction) -> AgentRunRecord (status=completed, result, trace)

Non-streaming : AgentOrchestrator.run_agent (boucle complète). Streaming : AgentOrchestrator.stream_response
ATTENTION : outils non exécutés aujourd'hui (sélection uniquement, voir ROADMAP P2 #4).

Points d'entree : `api/services/agent_orchestrator.py`, `api/services/agent_tools.py`,
`api/services/agent_long_term_memory.py`.

4. Workflows

Trigger (webhook / schedule / manual) -> WorkflowRun cree (status=pending) -> tache Celery dispatchee -> execution sequentielle des blocs :

- code (expression AST eval)
- http (requete externe)
- llm (appel LLM)
- rag (retrieval + generation)
- condition (branchement)
- human (approbation)
- email, calendar, database...

-> WorkflowRun.status = completed + output

Points d'entree : `api/services/workflow_engine.py`, `api/tasks/workflow_runs.py`, `api/services/workflow_blocks/`.

Flux transverses

Notifications

Evenement (billing, workflow, member, quota...) -> `api/services/notifications.py` -> NotificationTemplate (DB override ou code default) -> Notification (in-app) -> NotificationDelivery (email, si configure) -> envoi Twilio (SMS/WhatsApp) si active

Billing

Evenement provider (Stripe/Paystack webhook) -> verification signature -> `handle_stripe_webhook` / `handle_paystack_webhook` -> PaymentEvent (idempotent) -> mise a jour Subscription -> notification (`payment_failed`, `payment_succeeded`...) -> debit/credit credits (AI Pack)

Multi-tenancy

Chaque requete porte un `org_id`. Les checks sont doubles :

- Applicatif : `require_permission("resource:action")`
- DB : RLS PostgreSQL (`org_id = current_setting(...)`)

Voir SECURITY.md.